

0.31.1. Deux lettres à la fois — de base, ailleurs

Le 0.31 déplacé : le A part de la colonne 10, le B descend en colonne 10.

1. Ce qu'il faut comprendre

C'est le **0.31**, avec une autre lettre, le **B** (`ALPHABET[1]`, la 2e lettre), et une autre place : la **colonne 10** : le A part de (10, 0) vers la droite, le B de (10, 2) vers le bas.

Leurs chemins ne se croisent pas : le A reste sur la ligne 0, le B commence à la ligne 2.

C'est la version de base : une seule lettre, à la deuxième place. Le 0.31.2 met les deux ensemble.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <un_pas>` → [0.29.4. La fonction un_pas\(\) — un seul pas, sans attendre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1124 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      74 o ← la console, à l'allumage — la mise en route de la console
un_pas()       117 o ← main() — une fonction fournie par le langage
main()         111 o ← démarrage — ta fonction
VBlank         8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AttendreVBlank 16 o ← démarrage — attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases — attend de pouvoir écrire une case
AdresseCase    10 o ← effacer(), poser() — trouve l'adresse d'une case de l'écran
EffacerCases   12 o ← effacer() — efface des cases
CopierTuiles   18 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage — vide les deux cartes de l'écran
Tuiles        704 o ← CopierTuiles — la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser() × 3    39 o sur place, dans un_pas(), main() + AdresseCase (10 o)
effacer() × 1  15 o sur place, dans un_pas() + AdresseCase (10 o), EffacerCases (12 o)
image() × 1    3 o sur place, dans main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
un_pas() × 2   44 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
#include <un_pas>      // fait faire un seul pas à une tuile
```

2. UNE VARIABLE — XA

```
uint8_t xa = 10;    // la colonne du A
```

3. UNE VARIABLE — YA

```
uint8_t ya = 0;     // sa ligne
```

4. UNE VARIABLE — XB

```
uint8_t xb = 10;    // la colonne du B
```

5. UNE VARIABLE — YB

```
uint8_t yb = 2;     // sa ligne
```

6. UNE VARIABLE — IMAGES

```
uint8_t images = 0; // le chronomètre
```

7. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

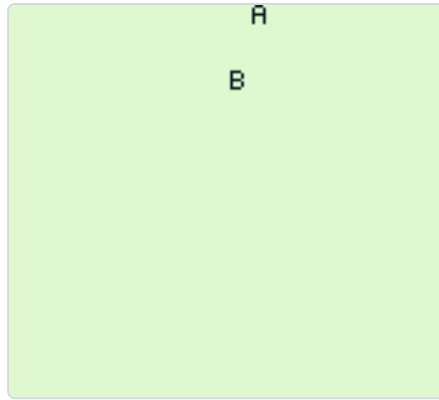
```
int main() {
    poser(xa, ya, ALPHABET[0]);
    poser(xb, yb, ALPHABET[1]);
    while (true) {
        image();
        images++;
        if (images == 15) {
            images = 0;
            un_pas(xa, ya, ALPHABET[0], 1, 0);
            if (yb < 16) un_pas(xb, yb, ALPHABET[1], 0, 1);
        }
    }
}
```

3. Ce qui se passe, étape par étape

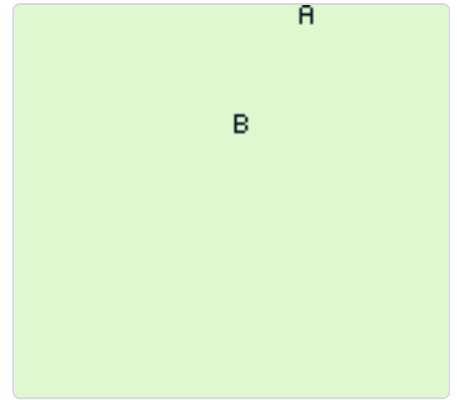
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



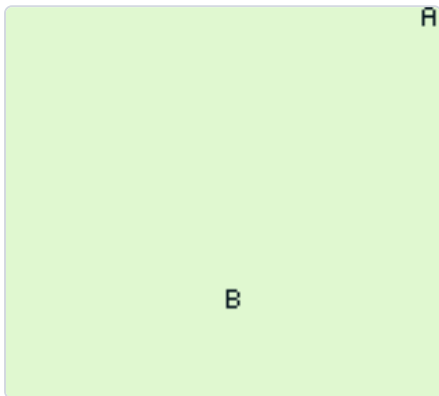
la toute première image — la console vient de démarrer — image 1



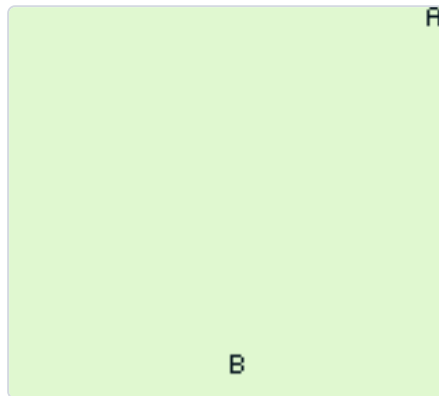
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un A qui file à droite depuis le milieu de la ligne du haut, un B qui descend en colonne 10.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A va jusqu'au bord : 19 ($x_a = 19$)

✓ le B s'arrête en ligne 16 ($y_b = 16$)

Ensuite : **0.31.2. Deux lettres à la fois — doublé, deux positions** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++